



CHALMERS
UNIVERSITY OF TECHNOLOGY

CA 2 - Contact Mechanics

TME250 - Finite Element Method: Solids

Academic year 2025/2026, SP2

Sankalp Manojkumar Prabhu, sankalpm@chalmers.se

December 9, 2025

Contents

1	Task 1	1
1.1	Problem Description	1
1.2	Methodology	1
1.3	Result and Analysis	2
2	Task 2	4
2.1	Problem Description	4
2.2	Methodology	4
2.2.1	Implicit Dynamics Without Contact	5
2.2.2	Implicit Dynamics With Contact	5
2.2.3	Explicit Dynamics With Contact	5
2.3	Result and Analysis	5
2.3.1	Implicit Dynamics Without Contact	6
2.3.2	Implicit Dynamics With Contact	7
2.3.3	Explicit Dynamics With Contact	9
2.3.3.1	Case with Large Time Step	9
2.3.3.2	Case with sufficiently Small Time Step	10
2.4	Discussion	12
2.4.1	Contact response	12
2.4.2	Energy Evolution	13
2.4.3	Contact Force	13
2.4.4	Advantages of Implicit and Explicit Simulation	13
2.4.5	Recognition of Insufficient Time Step	13
2.4.6	Interaction between Mesh Size and Time Step	13
A	Appendix: Scenario 1	I
A.1	Quasi-Static Contact	I
B	Appendix: Scenario 2	VII
B.1	Implicit Dynamic Without Contact	VII
B.2	Implicit Dynamic With Contact	XI
B.3	Explicit Dynamic	XVII
C	Appendix: Supporting Function	XXIII
C.1	Gap Function	XXIII

1

Task 1

1.1 Problem Description

In this task, a two dimensional domain as shown in figure 1.1 is modelled under plane strain condition with symmetric boundary condition on left and bottom edges. A circular cylinder of radius 40 mm, assumed to be rigid, is placed with its center at (0,60) mm such that it is in contact with top left node of the elastic block. The cylinder is then incrementally pushed downward to final position of (0,55) mm.

The contact between the cylinder and elastic block is computed using Lagrange multiplier method along with active set iteration to identify the active contact nodes at each load step. The vertical reaction force is computed throughout the indentation process from the sum of Lagrange multipliers. At the final step, the von Mises stress distribution is evaluated and presented in the result section.

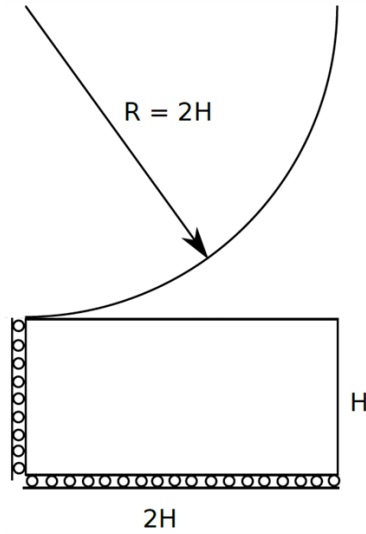


Figure 1.1: Problem Description

1.2 Methodology

The quasi-static contact simulation was implemented using a finite element method of the elastic block with a uniform triangular mesh generated by the provided "*Mesh.m*" script.

At every increment of cylinder moving from (0,60) to (0,55) mm, the contact between the cylinder and the block was computed through Lagrange multiplier formulation. For all the nodes on the top surface, the gap function relative to current cylinder position was computed and nodes with negative gap values were marked as active nodes. An active

set while loop was used to update the set of active nodes until convergence, ensuring that only nodes in compression contributed to the contact constraint.

Global stiffness matrix for the block was assembled using *CALFEM* for standard CST element. For each contact node the linearized contact normal N_o and initial gap g_o were computed, linearizing around previously converged solution to improve accuracy. The global system for lagrange multipliers is as shown below:

$$\begin{bmatrix} K & {}^{n-1}C_C^T \\ {}^{n-1}C_C & 0 \end{bmatrix} \begin{bmatrix} {}^n u \\ {}^n \lambda \end{bmatrix} = \begin{bmatrix} {}^n f \\ -{}^{n-1}G_C + {}^{n-1}C_C {}^{n-1}a \end{bmatrix} \quad (1.1)$$

The vertical reaction force on the cylinder was computed by summing the vertical component of all active Lagrange multipliers. At the last time step, the von Mises stress distribution and displacement field were extracted for visualization.

1.3 Result and Analysis

Figure 1.2 shows the vertical reaction force on the rigid cylinder as a function of indentation depth. The force-indentation relationship is approximately linear, increasing from zero at initial contact to approximately 180 kN at the final penetration at the 5mm penetration depth.

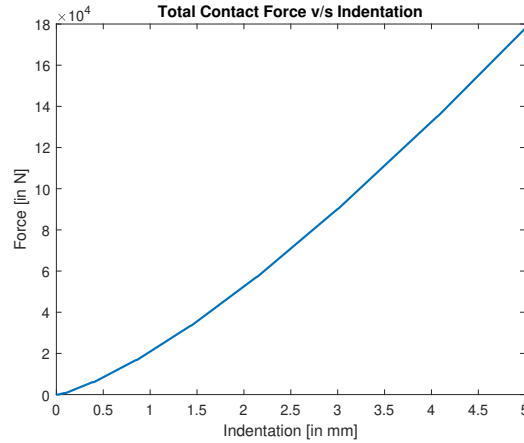


Figure 1.2: Force v/s Indentation

At the final indentation step, the von Mises stress distribution within the block is presented in figure 1.3. The highest stress occur directly beneath the rigid cylinder.

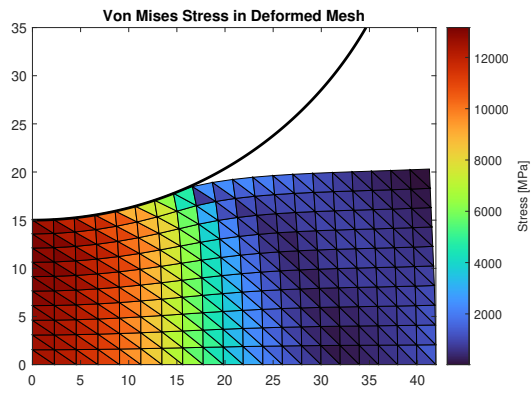


Figure 1.3: Von Mises Stress

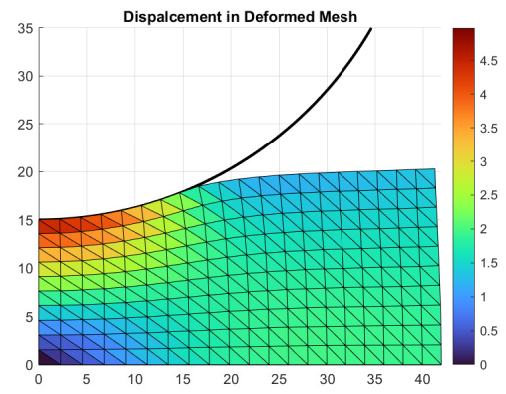


Figure 1.4: Displacement

The displacement field showed in figure 1.4 shows expected vertical compression under the cylinder.

2

Task 2

2.1 Problem Description

In the second task, the behavior of the elastic block is evaluated under dynamic loading. Unlike Task 1, the rigid cylinder remains fixed in its initial position at (0,60) mm, while elastic block is assigned with an initial upward velocity.

The scenario is analyzed in three stages. First, block is simulated dynamically without contact, using a vertical displacement constraint on the top boundary. Next, the constraint is replaced with the cylinder, and dynamic contact is introduced using the same Lagrange multiplier contact formulation from Task 1. Finally, the simulation is repeated with an explicit time integration.

Snapshots of the deformation, contact force history and the evolution of internal, kinetic and total energy are obtained for both implicit and explicit simulation.

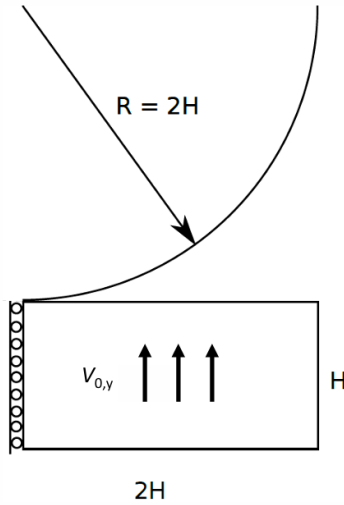


Figure 2.1: Problem Description

2.2 Methodology

The dynamic analysis was implemented by extending the code from Task 1 to include dynamic contact. The block was assigned an initial vertical velocity. For dynamic simulation, mass matrix is also needed. A consistent mass matrix can be obtained by exact integration of the shape functions over the element domain. But this leads to a fully populated global mass matrix. For explicit time integration, repeated inversion of full matrix is highly inefficient. Therefore a lumped mass matrix approximation is adopted,

in which element mass is directly distributed over nodal degree of freedom, resulting in a diagonal mass matrix.

2.2.1 Implicit Dynamics Without Contact

In this case the top boundary of the block was constrained in vertical direction, preventing movement. The displacement field as well as internal, kinetic and total energy was computed.

2.2.2 Implicit Dynamics With Contact

The top constraint was removed and replaced by contact with stationary cylinder. At every time step, contact nodes were identified using the same Lagrange multiplier formulation with active set strategy from Task 1, but now integrated with dynamic solver. Contact forces, deformation snapshots and energy measures were collected until the block impacted the cylinder and rebounded.

For Implicit Dynamics below is the system of equations that is solved.

$$\left[\frac{2}{(\Delta t)^2} \underline{M} + \frac{1}{2} \underline{S} \right] {}^n \underline{a} + \underline{C}_C ({}^{n-1} \underline{a})^T \underline{\Lambda} = \underline{f}_{\text{ext}} \left(t_{n-1} + \frac{\Delta t}{2} \right) + \frac{2}{\Delta t} \underline{M} {}^{n-1} \underline{v} + \left[\frac{2}{(\Delta t)^2} \underline{M} - \frac{1}{2} \underline{S} \right] {}^{n-1} \underline{a} \quad (2.1)$$

$$\underline{C}_C ({}^{n-1} \underline{a}) {}^n \underline{a} = -\underline{G}_C ({}^{n-1} \underline{a}) + \underline{C}_C ({}^{n-1} \underline{a}) {}^{n-1} \underline{a} \quad (2.2)$$

$${}^n \underline{v} = \frac{2}{\Delta t} \left[{}^n \underline{a} - {}^{n-1} \underline{a} \right] - {}^{n-1} \underline{v} \quad (2.3)$$

2.2.3 Explicit Dynamics With Contact

For explicit simulation the time stepping scheme was changed to the explicit $\theta = 0$ variant of the midpoint rule from equation 10.133 from the compendium.

Two simulations were run: one with a deliberately large time step to demonstrate instability and another with a sufficiently small, CFL-compliant time-step. As in the implicit case, the deformation, contact force and energy evaluation were extracted for comparison.

For Explicit Dynamics below is the system of equations that is solved.

$$\left[\frac{(\Delta t)^2}{2} \underline{C}_C \underline{M}_L^{-1} \underline{C}_C^T \right] \underline{\Lambda} = \underline{G}_C ({}^{n-1} \underline{a}) + \underline{C}_C \left[{}^n \underline{a}^{\text{tr}} - {}^{n-1} \underline{a} \right] \quad (2.4)$$

$${}^n \underline{a}^{\text{tr}} = {}^{n-1} \underline{a} + \Delta t {}^{n-1} \underline{v} + \frac{(\Delta t)^2}{2} \underline{M}_L^{-1} \left[\underline{f}_{\text{ext}}(t_{n-1}) - \underline{S} \underline{a} \right] \quad (2.5)$$

$${}^n \underline{v} = \frac{2}{\Delta t} \left[{}^n \underline{a} - {}^{n-1} \underline{a} \right] - {}^{n-1} \underline{v} \quad (2.6)$$

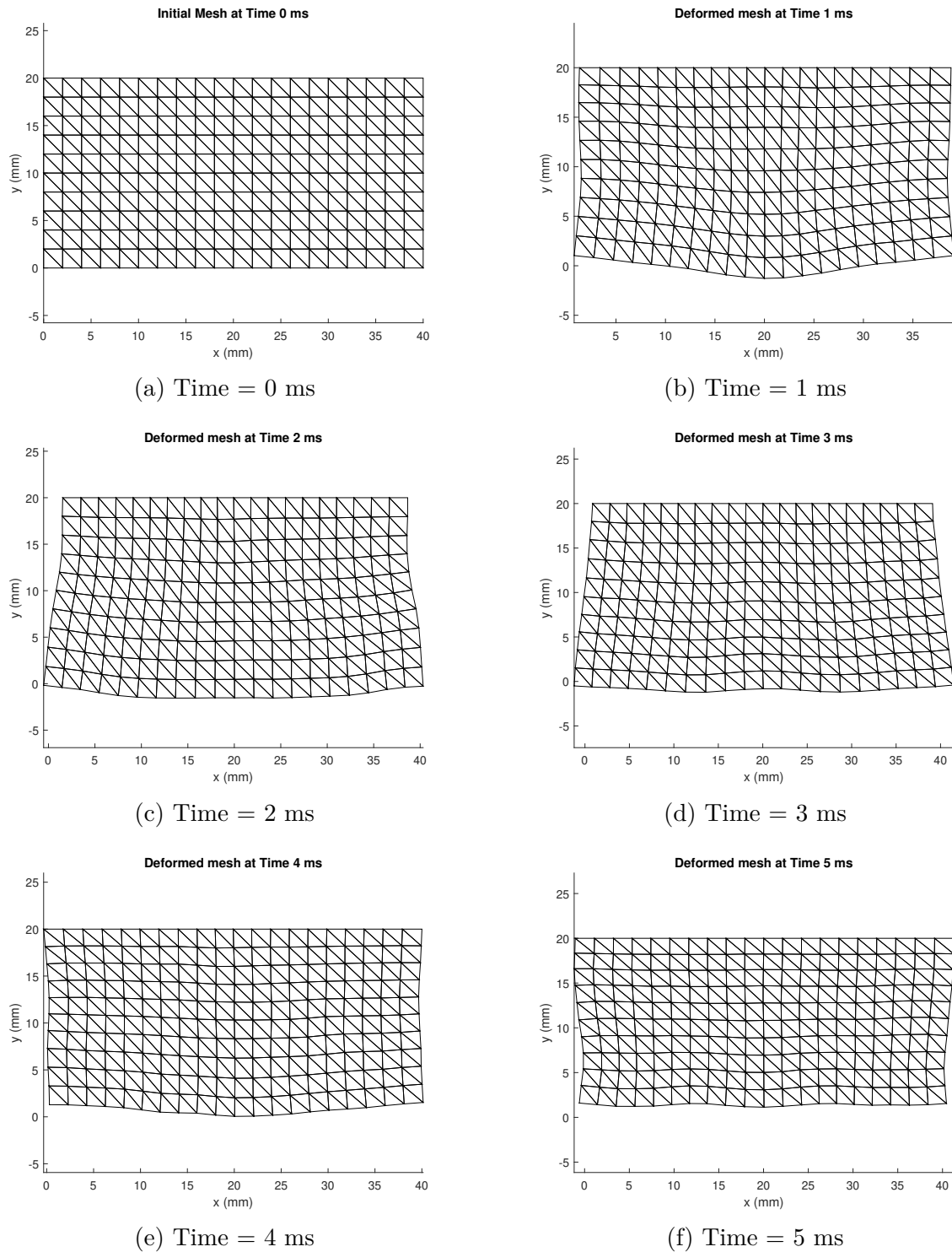
2.3 Result and Analysis

This section shows the result for implicit and explicit dynamic contact scenarios

2.3.1 Implicit Dynamics Without Contact

Figure 2.2 shows deformation for selected timesteps from the implicit dynamic simulation with top boundary vertically constrained. After assigning the initial upward velocity, the block deforms and shows symmetric deformation and oscillatory motion, which is expected in this case.

Figure 2.2: Deformation of Block for Selected Time Step



The internal, kinetic and total energy histories are presented in figure 2.3. The kinetic energy and internal strain energy are oscillating out of phase, reflecting continuous exchange between kinetic energy and internal strain energy. The total energy remains nearly constant throughout simulation which is expected because the system is modeled under ideal conditions with no dampening mechanism.

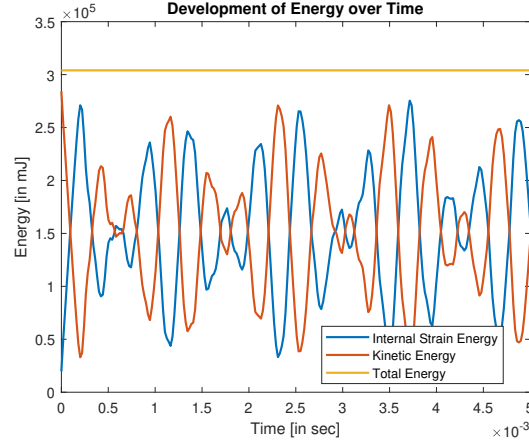
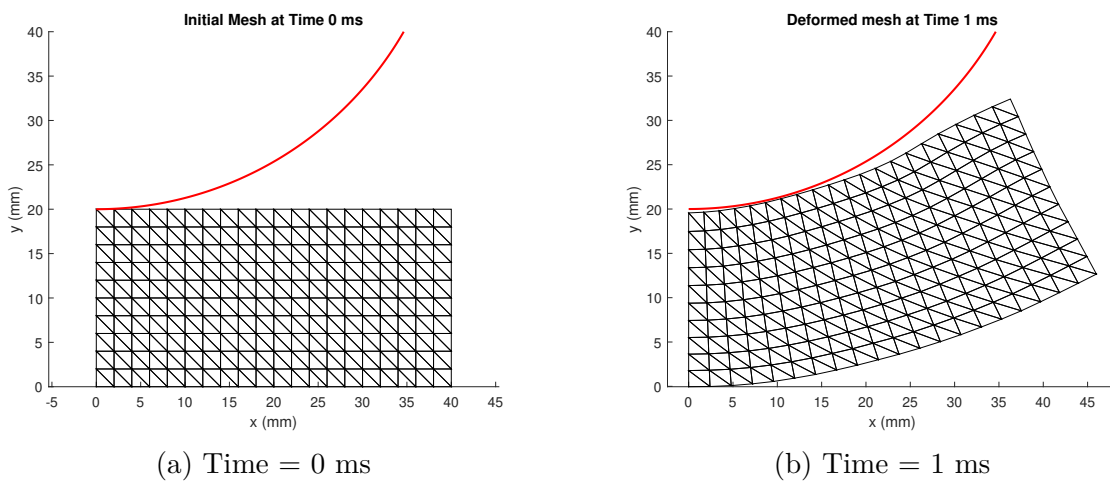


Figure 2.3: Evolution of Energy over Time

2.3.2 Implicit Dynamics With Contact

Figure 2.4 shows the deformation of the block during collision with the rigid cylinder. As the block moves upward the upper surface comes in contact with the cylinder. The contact region progressively engages more nodes reaching the maximum compression. Following the impact, the block rebounds and separate from the cylinder. Two more contact-separation cycles are observed with decreasing amplitude.

Figure 2.4: Deformation of Block for Selected Time Step in Implicit Dynamics



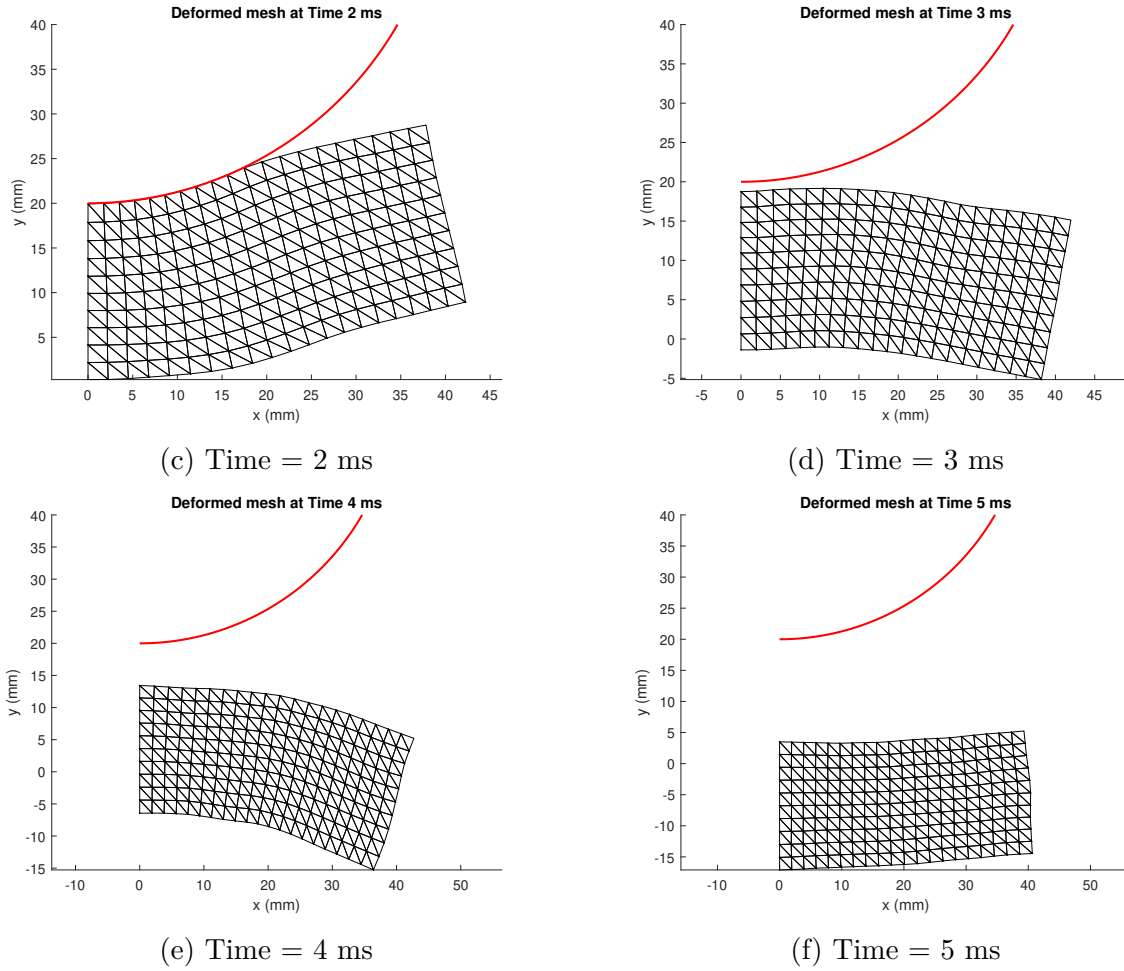


Figure 2.5 shows the vertical contact force history on the cylinder. The force exhibits peak compression force of 142.5 kN during maximum penetration. Two more peaks were also observed as the block bounces, with decreasing peak force.

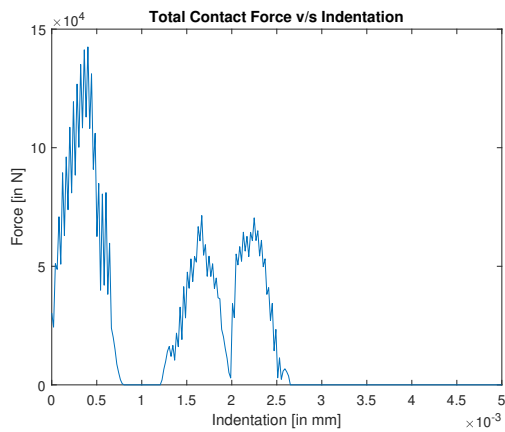


Figure 2.5: Force v/s Indentation for Implicit Dynamics

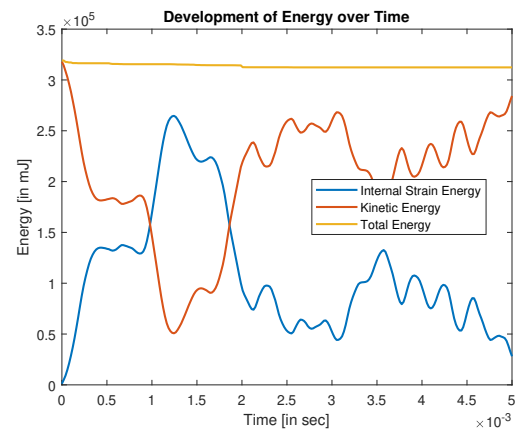


Figure 2.6: Evolution of Energy over Time for Implicit Dynamics

Figure 2.6 shows the energy evolution in time. Internal strain energy and kinetic energy oscillate out of phase during free motion. During contact event, strain energy increases corresponding to compression of block, with simultaneous decrease in kinetic energy. The

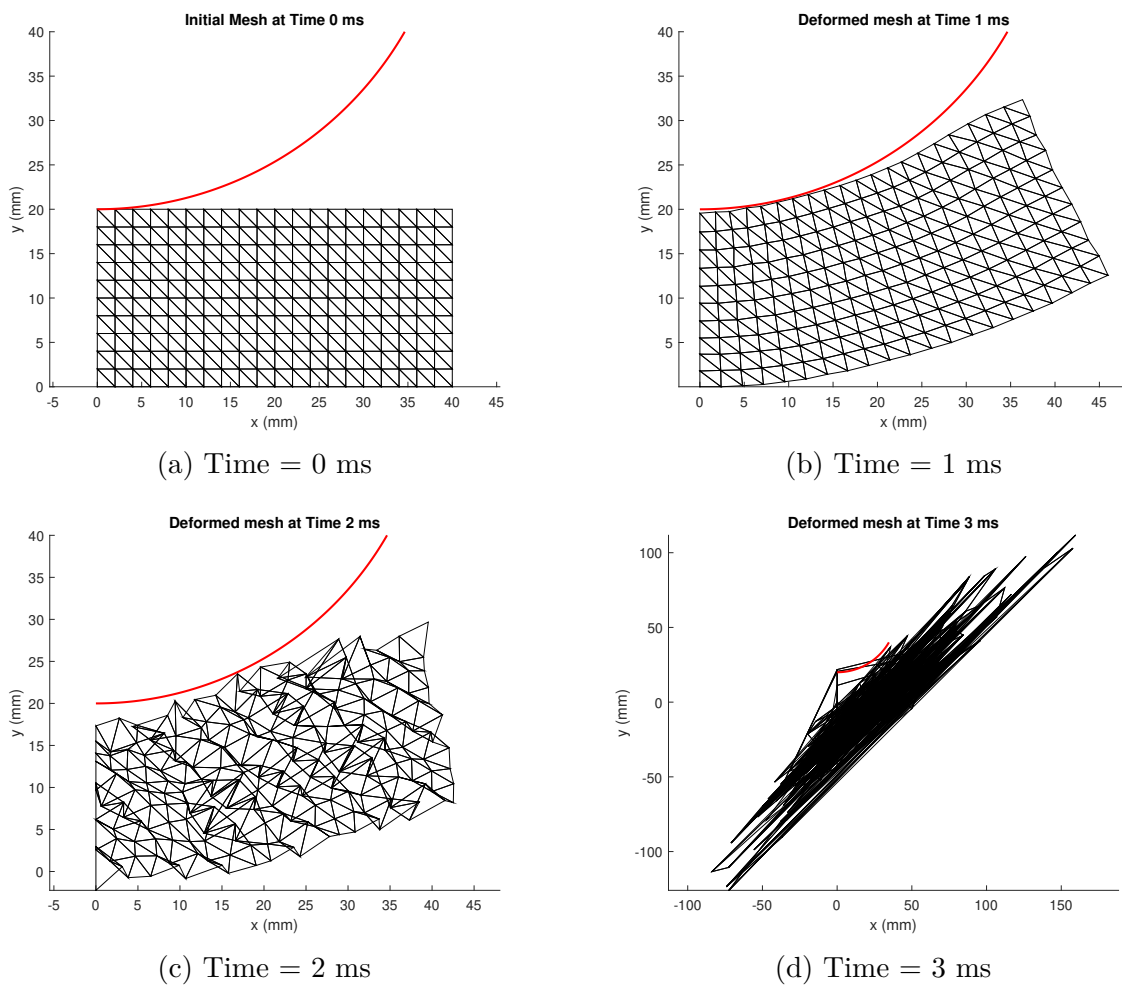
total energy remains approximately constant with a very negligible loss attributing to numerical approximations.

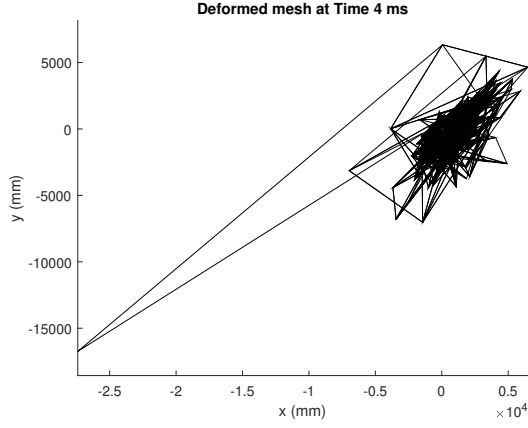
2.3.3 Explicit Dynamics With Contact

2.3.3.1 Case with Large Time Step

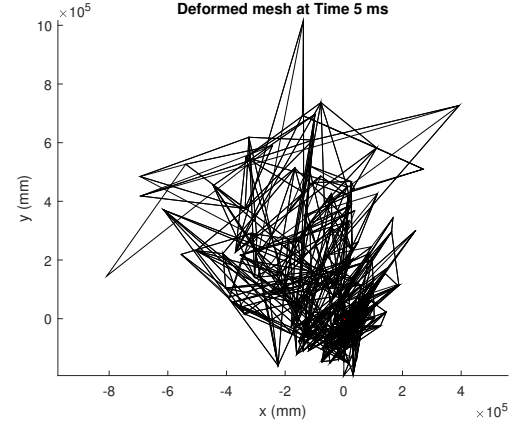
Figure 2.7 shows the simulation result with $\Delta t = 1e - 6$ s. The deformation plot reveals severe mesh distortion after initial contact, with nodes exhibiting non-physical displacement. The block penetrates the cylinder, indicating that the contact algorithm cannot respond quickly enough to prevent penetration.

Figure 2.7: Deformation of Block for Selected Time Step in Explicit Dynamics for Larger Time Step





(e) Time = 4 ms



(f) Time = 5 ms

Figure 2.8 shows the energy evolution for the unstable case. The total energy grows exponentially reaching values of several orders of magnitude above the initial energy. The energy explosion is an indicator of numerical instability

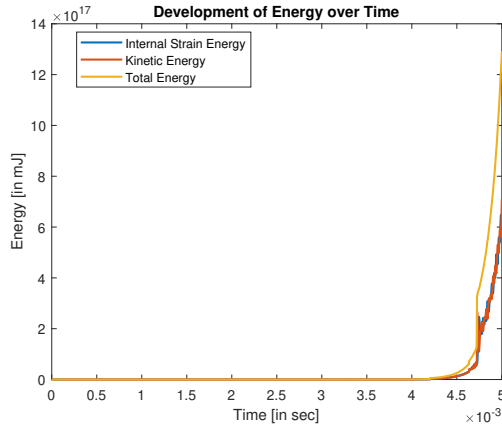


Figure 2.8: Evolution of Energy over Time for Explicit Dynamics For Large Time Step

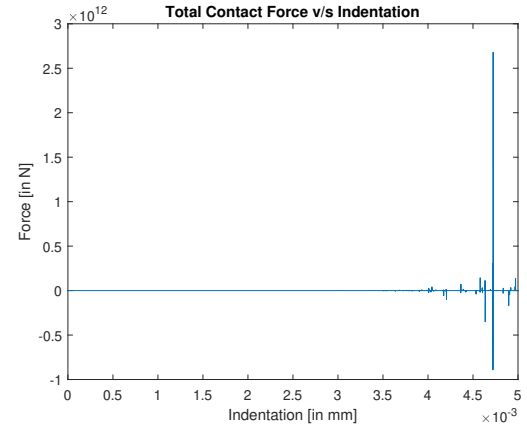


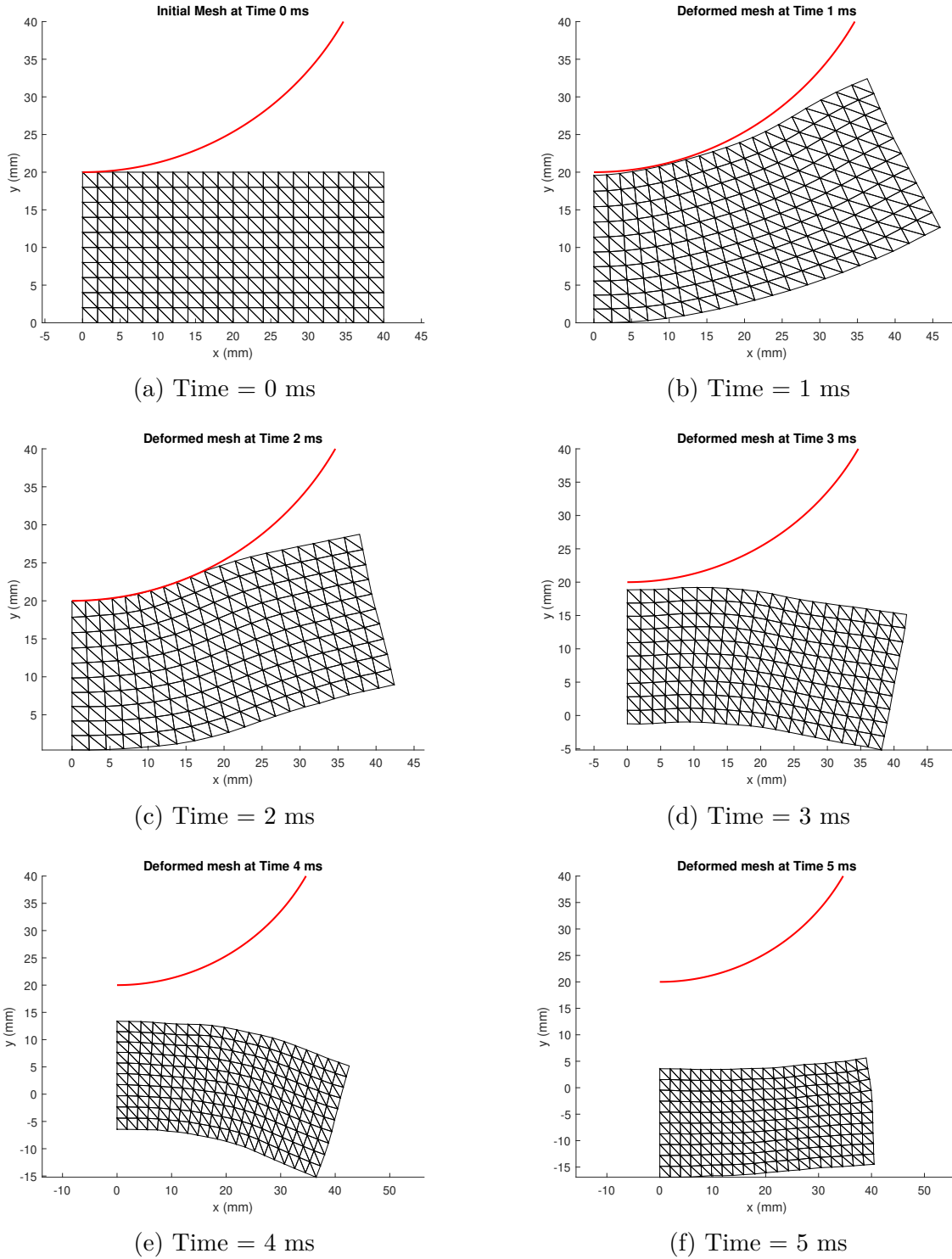
Figure 2.9: Force v/s Indentation for Explicit Dynamics For Large Time Step

The contact force history in figure 2.9 also shows unrealistic magnitudes of peak forces which confirms that the explicit dynamics cannot handle the contact with larger timestep.

2.3.3.2 Case with sufficiently Small Time Step

For the stable explicit simulation, a time step of $\Delta t = 1e-7$ s was used, which is approximately 200 times smaller than the implicit simulation and well below CFL limit. Figure 2.10 shows the deformation snapshots, which qualitatively match the implicit results.

Figure 2.10: Deformation of Block for Selected Time Step in Explicit Dynamics for sufficiently smaller time step



The contact force history in figure 2.11 shows a higher instantaneous peaks and higher frequency oscillation in values than implicit result. This behavior is attributed to the low numerical dampening of the explicit simulation ($\theta = 0$) compared to implicit simulation using Crank Nicolson scheme ($\theta = 0.5$) which averages the stiffness contribution over the time step resulting in dampening of high frequency force.

But by applying the "*smoothdata*" filter in Matlab, the noisy force data from ex-

licit simulation is smoothened by moving average and the force magnitude and contact-separation cycle matches with the implicit solution as shown in figure 2.12.

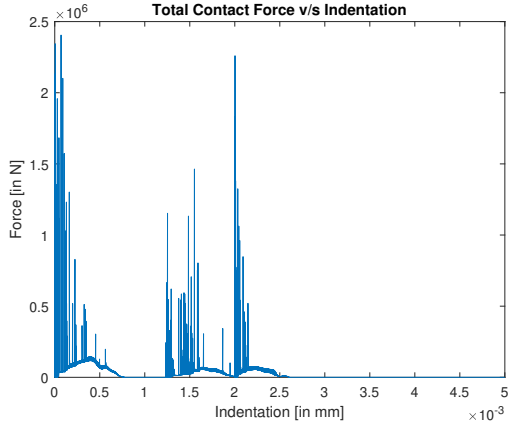


Figure 2.11: Force v/s Indentation for Explicit Dynamics for sufficiently smaller Time Step

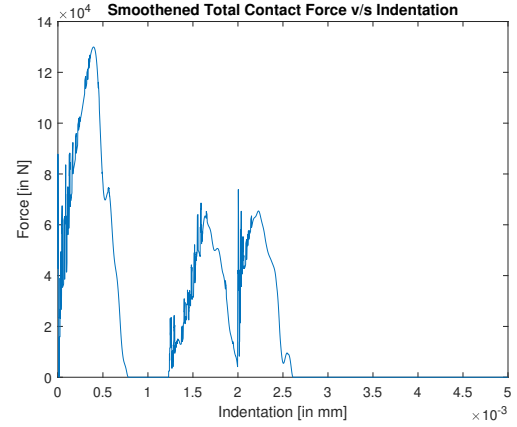


Figure 2.12: Smoothened Force v/s Indentation for Explicit Dynamics for sufficiently smaller Time Step

Figure 2.13 shows the energy evolution in time. Internal strain energy and kinetic energy oscillate out of phase during free motion. This plot closely matches with figure 2.6 as in implicit dynamics case confirming the stability and accuracy of simulation when time-step is sufficiently small.

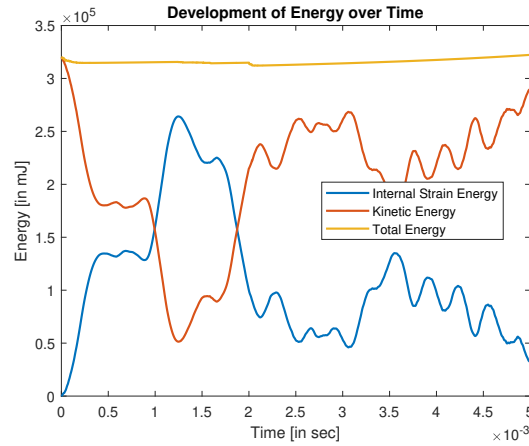


Figure 2.13: Evolution of Energy over Time for Explicit Dynamics for sufficiently small Time Step

2.4 Discussion

Dynamic simulation in scenario 2 demonstrate the transient response of the elastic block during impact with a rigid cylinder.

2.4.1 Contact response

When the block is given an initial upward velocity, it impacts the stationary cylinder, which leads to rapid increase in contact force. After the contact the block rebounds and

undergoes oscillatory motion leading to two more smaller impacts before falling. This behavior is consistent in both implicit and explicit simulation.

2.4.2 Energy Evolution

The kinetic and internal strain energy oscillate out of phase, while the total energy remains nearly constant throughout the simulation in both implicit and explicit simulation with sufficiently small timestep. This confirms that the system is modelled under ideal condition.

But in case where the time step is large in explicit simulation nonphysical energy growth can be observed which is a mark of instability in the solution.

2.4.3 Contact Force

The explicit simulation exhibits higher contact force peaks compared to implicit simulation. This difference arises from the integration schemes used in both implicit and explicit simulation. But when force contact from explicit is smoothed using moving average, the resulting force history closely matches with implicit simulation.

Explicit simulation for larger timestep shows unrealistic magnitudes of peak force which also confirms that explicit simulation showcases instability with larger timestep.

2.4.4 Advantages of Implicit and Explicit Simulation

The implicit simulation offers improved stability and allows larger timestep without loss of stability, making it well suited for moderately dynamic problems. However, it requires solving a coupled system of equations at each time step, increasing computational cost.

The explicit simulation is computationally efficient per time step as it solves considerably smaller system of equations. It is particularly attractive for impact-dominated problems. Its main limitation is the strict stability constraint on time step, which must satisfy CFL condition.

2.4.5 Recognition of Insufficient Time Step

An insufficiently small time step in explicit simulation can be identified by

- Rapid growth of Total Energy.
- Non-physical deformation pattern.
- Unrealistic force magnitudes.

2.4.6 Interaction between Mesh Size and Time Step

In explicit simulation, the stable time step is directly related to smallest element size in the mesh. Refining mesh increases the total number of time step required for stability. As a result, explicit simulation are highly sensitive to mesh resolution.

A

Appendix: Scenario 1

A.1 Quasi-Static Contact

```
1 clear all
2 close all
3 clc
4
5 %% Material Properties
6
7 E = 50e3; %Mpa
8 nu = 0.3;
9 rho = 8e-6;%kg/mm3
10 T = 5e-3;%sec
11 dt = 20e-6;%sec
12 v_ini = 10e3;%mm/sec
13 edge_length = 2;%mm
14 thickness = 1; % Assumed
15 R = 40;
16 H = 20;%mm
17 L = 2*H;%mm
18
19 %% Meshing
20 ny = H/edge_length;
21 order = 1;
22 [coords, edges, triangle] = mesh(ny,order,false);
23 coords = 1000.*coords;%mm
24 Ex = coords(1,:);
25 Ey = coords(2,:);
26 nnodes = size(coords,2);
27 nelem = size(triangle,2);
28 Edof = zeros(nelem, size(triangle,1)*2+1);
29
30 for i = 1:nelem
31
32     dofx = 2*triangle(:,i)-1;
33     dofy = 2*triangle(:,i);
34     Edof(i,1)=i;
35     Edof(i,2:2:end) = dofx;
36     Edof(i,3:2:end) = dofy;
37
38 end
39
40 %% Element Routine
41
42 ptype = 2;
43 ep = [ptype thickness];
```



```

44 ndof = 2*nnodes;
45 D = hooke(ptype,E,nu);
46 S = sparse(ndof,ndof);
47 F = zeros(ndof,1);
48
49 for ielem = 1:size(Edof,1)
50
51     enodes = triangle(:,ielem);
52     ex = Ex(enodes);
53     ey = Ey(enodes);
54     [Ke,fe] = plante(ex,ey,ep,D);
55     edof = Edof(ielem,2:end);
56     S(edof,edof) = S(edof,edof)+Ke;
57     F(edof) = F(edof)+fe;
58
59 end
60
61 %% Boundary Conditions
62
63 fixed_dofs = [];
64 left_edge = find(edges(end,:)==5);
65
66 for le = 1:size(left_edge,2)
67     n1 = edges(1,left_edge(1,le));
68     n2 = edges(2,left_edge(1,le));
69     fixed_dofs = [fixed_dofs,2*n1-1,2*n2-1];
70 end
71
72 bottom_edge = find(edges(end,:)==1);
73
74 for be = 1:size(bottom_edge,2)
75     n1 = edges(1,bottom_edge(1,be));
76     n2 = edges(2,bottom_edge(1,be));
77     fixed_dofs = [fixed_dofs,2*n1,2*n2];
78 end
79
80 fixdof = unique(fixed_dofs);
81
82 %% Cylinder movement
83
84 y_ini = 60;
85 y_final = 55;
86 n_s = 250; %Number of steps
87 y = linspace(y_ini,y_final,n_s);
88 X_bar = zeros(2,n_s);
89 X_bar(1,:) = 0;
90 X_bar(2,:) = y;
91
92 %% Defining Contact Nodes
93
94 Contact_id = [3,4];
95 top_right_edge = find(edges(end,:)==Contact_id(1));
96 top_left_edge = find(edges(end,:)==Contact_id(2));
97 top_right_nodes = reshape(edges(1:end-1,top_right_edge),1,[]);
98 top_left_nodes = reshape(edges(1:end-1,top_left_edge),1,[]);
99 C_nodes = unique([top_right_nodes,top_left_nodes]);
100 num_candidate = numel(C_nodes);
101 active_set = false(num_candidate,1);

```

```

102
103 %% Pre-Initiation before the step loop
104
105 u0 = zeros(ndof,1);
106 force_hist = zeros(n_s,1);
107 indentation = linspace(0,y_ini-y_final,n_s);
108 g0 = zeros(num_candidate,1);
109 NO = zeros(2,num_candidate);
110
111 for ic = 1:num_candidate
112
113     ni = C_nodes(ic);
114     xi = coords(:,ni);
115     u_prev = [u0(2*ni-1);u0(2*ni)];
116     [g0(ic),NO(:,ic)] = gap_lin(X_bar(:,1),xi,R,u_prev);
117
118 end
119
120 %% Loading Loop
121
122 for n= 2:n_s+1
123     x_c = X_bar(:,n-1);
124     active_local=active_set;
125     bool = true;
126     k= 0 ;
127     while k ==0 || bool
128
129         nC = sum(active_local);
130         index = find(active_local);
131         C_c = zeros(nC,ndof);
132         G_c = zeros(nC,1);
133         node_list = zeros(nC,1);
134         if nC~=0
135             for i = 1:nC
136
137                 idx = index(i);
138                 ni = C_nodes(idx);
139                 C_i = zeros(1,ndof);
140                 C_i(1,2*ni-1)= NO(1,idx);
141                 C_i(1,2*ni) = NO(2,idx);
142                 C_c(i,:) = C_i;
143                 G_c(i) = g0(idx);
144                 node_list(i)= ni;
145
146             end
147
148             total_dof = ndof+nC;
149             u = zeros(total_dof,1);
150             freedof = setdiff((1:total_dof)',fixdof);
151             z = sparse(nC,nC);
152             K = [S , C_c';
153                 C_c , z];
154             Force = [F;-G_c+C_c*u0];
155             u(freedof)= K(freedof,freedof)\Force(freedof);
156             lambda = u(ndof+1:end);
157
158         else
159             u = zeros(ndof,1);

```

```

160         freedof = setdiff((1:ndof)', fixdof);
161         u(freedof) = S(freedof, freedof) \ F(freedof);
162         lambda = [];
163     end
164     k = k+1;
165     new_active_local = active_local;
166     if ~isempty(lambda)
167
168         for i = 1:nC
169             idx = index(i);
170
171             if lambda(i) <= 0
172
173                 new_active_local(idx) = true;
174             else
175                 new_active_local(idx) = false;
176             end
177         end
178     end
179     g_new = zeros(num_candidate, 1);
180     N_new = zeros(2, num_candidate);
181     for j = 1:num_candidate
182         ni = C_nodes(j);
183         xi = coords(:, ni);
184         u_g = [u(2*ni-1); u(2*ni)];
185         u_prev = [u0(2*ni-1); u0(2*ni)];
186         [gx, Nx] = gap_lin(x_c, xi, R, u_prev);
187         g_new(j) = gx + Nx' * u_g;
188         N_new(:, j) = Nx;
189         if g_new(j) < 0
190
191             new_active_local(j) = true;
192         end
193     end
194
195     bool = any(new_active_local ~= active_local);
196     active_local = new_active_local;
197 end
198 g0 = g_new;
199 N0 = N_new;
200 u0 = u(1:ndof);
201 active_set = new_active_local;
202
203 if ~isempty(lambda)
204
205     NL = zeros(numel(lambda), 1);
206
207     for il = 1:numel(lambda)
208
209         ni = node_list(il);
210         X = coords(:, ni);
211         u_node = [u(2*ni-1); u(2*ni)];
212         [~, N_node] = gap_lin(x_c, X, R, u_node);
213         NL(il) = N_node(2);
214
215     end
216     F_contact = sum(lambda .* NL);
217 else

```

```

218     F_contact = 0;
219     end
220
221     force_hist(n-1)= F_contact;
222 end
223
224 vm = zeros(size(Edof,1),1);
225 for ielem= 1:size(Edof,1)
226
227     enodes = triangle(:,ielem);
228     ex = Ex(enodes);
229     ey = Ey(enodes);
230     edof = Edof(ielem,2:end);
231     ed = u0(edof);
232     [es,et]= plants(ex,ey,ep,D,ed');
233     sigma = [es(1),es(4),0;
234             es(4),es(2),0;
235             0,0,es(3)];
236     sij = sigma - (1/3)*trace(sigma)*eye(3);
237     vm(ielem) = sqrt(3/2*sum(sum(sij.*sij)));
238
239 end
240
241 ed1 = extract_ed(Edof,u0);
242 Ex_plot = zeros(size(Edof,1),3);
243 Ey_plot = zeros(size(Edof,1),3);
244 for i = 1:size(Edof,1)
245
246     enodes = triangle(:,i);
247     Ex_plot(i,:)= coords(1,enodes);
248     Ey_plot(i,:)= coords(2,enodes);
249
250 end
251
252 %% Plotting
253
254 theta = linspace(-pi/6, -pi/2, 200);
255 xc = x_c(1); yc = x_c(2);
256 xarc = xc + R*cos(theta);
257 yarc = yc + R*sin(theta);
258
259 %Force v/s Indentation Plot
260 figure
261 plot(indentation,force_hist,LineWidth=1.5)
262 xlabel ('Indentation [in mm]')
263 ylabel ('Force [in N]')
264 title('Total Contact Force v/s Indentation')
265
266 %Von Mises Stress Plot
267 figure;
268 fill((Ex_plot(:,1:3)+ed1(:,1:2:6))', (Ey_plot(:,1:3)+ed1(:,2:2:6))', vm)
269 ; % Factor of Safety
270 hold on;
271 plot(xarc, yarc, 'k-', 'LineWidth', 2);
272 view(2);
273 axis equal tight;
274 colormap("turbo");
275 colorbar;

```

```
275 c = colorbar;
276 title('Von Mises Stress in Deformed Mesh');
277
278 % Displacement Plot
279 ux = u0(1:2:end);
280 uy = u0(2:2:end);
281 umag = hypot(ux, uy);
282 faces = triangle';
283 verts = coords';
284 verts_def = verts + [ux, uy];
285 figure;
286 trisurf(faces, verts_def(:,1), verts_def(:,2), umag);
287 hold on;
288 plot(xarc, yarc, 'k-', 'LineWidth', 2);
289 view(2);
290 axis equal tight;
291 colormap turbo;
292 cb = colorbar;
293 title('Dispalcement in Deformed Mesh');
```

B

Appendix: Scenario 2

B.1 Implicit Dynamic Without Contact

```
1 clear all
2 close all
3 clc
4
5 %% Material Properties
6
7 E = 50e3; %Mpa
8 nu = 0.3;
9 rho = 8e-6; %kg/mm3
10 T = 5e-3; %sec
11 dt = 20e-6; %sec
12 v_ini = 10e3; %mm/sec
13 edge_length = 2; %mm
14 thickness = 1; % Assumed
15 R = 40; %mm
16 H = 20; %mm
17 L = 2*H; %mm
18
19 %% Meshing
20 ny = H/edge_length;
21 order = 1;
22 [coords, edges, triangle] = mesh(ny,order,false);
23 coords = 1000.*coords;
24 Ex = coords(1,:);
25 Ey = coords(2,:);
26 nnodes = size(coords,2);
27 nelem = size(triangle,2);
28 Edof = zeros(nelem, size(triangle,1)*2+1);
29
30 for i = 1:nelem
31
32     dofx = 2*triangle(:,i)-1;
33     dofy = 2*triangle(:,i);
34     Edof(i,1)=i;
35     Edof(i,2:2:end) = dofx;
36     Edof(i,3:2:end) = dofy;
37
38 end
39
40 %% Element Routine
41
42 ptype = 2;
43 ep = [ptype thickness];
```

```

44 ndof = 2*nnodes;
45 D = hooke(ptype,E,nu);
46 S = sparse(ndof,ndof);
47 M = sparse(ndof,ndof);
48 F = zeros(ndof,1);
49
50 for ielem = 1:size(Edof,1)
51
52     enodes = triangle(:,ielem);
53     ex = Ex(enodes);
54     ey = Ey(enodes);
55     tri_coords = [ex;ey;1,1,1]';
56     Ae = 0.5*abs(det(tri_coords));
57     [Ke,~] = plante(ex,ey,ep,D);
58     Me = Ae*rho/3*eye(6);
59     edof = Edof(ielem,2:end);
60     S(edof,edof) = S(edof,edof)+Ke;
61     M(edof,edof) = M(edof,edof) + Me;
62
63 end
64
65 %% Boundary Conditions
66
67 fixed_dofs = [];
68 top_right_edge = find(edges(end,')==3);
69
70 for be = 1:size(top_right_edge,2)
71     n1 = edges(1,top_right_edge(1,be));
72     n2 = edges(2,top_right_edge(1,be));
73     fixed_dofs = [fixed_dofs,2*n1,2*n2];
74 end
75
76 top_left_edge = find(edges(end,')==4);
77
78 for be = 1:size(top_left_edge,2)
79     n1 = edges(1,top_left_edge(1,be));
80     n2 = edges(2,top_left_edge(1,be));
81     fixed_dofs = [fixed_dofs,2*n1,2*n2];
82 end
83
84 fixdof = unique(fixed_dofs);
85 freedof = setdiff((1:ndof)',fixdof);
86
87 %% Initiations before loop
88
89 v = zeros(ndof,1);
90 v(2:2:end)= v_ini;
91 v(fixdof)=0;
92 u = zeros(ndof,1);
93 n_s = 250;
94 U = zeros(ndof,n_s);
95 V = zeros(ndof,n_s);
96 int_st = zeros(n_s,1);
97 kin_eng = zeros(n_s,1);
98 T_s = linspace(0,T,n_s);
99
100 %% Time Loop
101

```

```

102 for i = 1:n_s
103
104     u_old = u;
105     v_old = v;
106     F_eff = F+((2/(dt^2))*M-0.5*S)*u_old+((2/dt)*M)*v_old;
107     S_eff = ((2/(dt^2))*M+0.5*S);
108     u(freedof) = S_eff(freedof,freedof)\F_eff(freedof);
109     U(:,i)= u;
110     v= (2/(dt))*(u-u_old)-v_old;
111     V(:,i)= v;
112     int_st(i,1)=0.5*u'*S*u;
113     kin_eng(i,1)=0.5*v'*M*v;
114 end
115
116 %% Plotting
117 % Simulation, Comment it for faster Run
118 scale=1;
119 vert = coords';
120 Face = triangle';
121 figure
122 h.patch=patch('Faces', Face, 'Vertices', vert,'FaceColor','none','
    EdgeColor',[0 0 0], 'LineWidth',0.5);
123 axis equal;
124 xlabel('x (mm)'); ylabel('y (mm)');
125 title('Deformed mesh (edges only)');
126 drawnow;
127
128 for k= 1:n_s
129
130     uk = U(:,k);
131     ux = uk(1:2:end);
132     uy = uk(2:2:end);
133     vertk= coords'+scale.*[ux, uy];
134     set(h.patch,'Vertices',vertk);
135     title(sprintf('Time step %d / %d', k, n_s));
136     drawnow;
137
138 end
139
140 % Energy over Time
141 figure
142 plot(T_s,int_st,LineWidth=1.5,DisplayName='Internal Strain Energy');
143 hold on
144 plot(T_s,kin_eng,LineWidth=1.5,DisplayName='Kinetic Energy');
145 plot(T_s,int_st+kin_eng,LineWidth=1.5,DisplayName='Total Energy');
146 xlabel ('Time [in sec]')
147 ylabel ('Energy [in mJ]')
148 legend (Location="best")
149 title('Development of Energy over Time')
150
151 %Deformation of Block for Selected Time Step
152 time_step =round([1e-3/dt,2e-3/dt,3e-3/dt,4e-3/dt,5e-3/dt]);
153 scale=1;
154 vert = coords';
155 Face = triangle';
156 figure
157 patch('Faces', Face, 'Vertices', vert,'FaceColor','none','EdgeColor',[0
    0 0], 'LineWidth',0.5);

```



```
158 axis equal;
159 xlabel('x (mm)'); ylabel('y (mm)');
160 title ('Initial Mesh at Time 0 ms')
161
162 for i=1:numel(time_step)
163
164     k = time_step(i);
165     uk = U(:,k);
166     ux = uk(1:2:end);
167     uy = uk(2:2:end);
168     vertk= coords'+scale.*[ux, uy];
169     figure
170     patch('Faces', Face, 'Vertices', vertk, 'FaceColor', 'none', 'EdgeColor',
171         '[0 0 0]', 'LineWidth', 0.5);
172     axis equal;
173     xlabel('x (mm)'); ylabel('y (mm)');
174     title (sprintf('Deformed mesh at Time %d ms',k*dt*1000))
175 end
```

B.2 Implicit Dynamic With Contact

```

1 clear all
2 close all
3 clc
4
5 %% Material Properties
6
7 E = 50e3; %Mpa
8 nu = 0.3;
9 rho = 8e-6; %kg/mm3
10 T = 5e-3; %sec
11 dt = 20e-6; %sec
12 v_ini = 10e3; %mm/sec
13 edge_length = 2; %mm
14 thickness = 1; % Assumed
15 R = 40; %mm
16 x_c = [0;60];
17 H = 20; %mm
18 L = 2*H; %mm
19
20 %% Meshing
21
22 ny = H/edge_length;
23 order = 1;
24 [coords, edges, triangle] = mesh(ny,order,false);
25 coords = 1000.*coords;
26 Ex = coords(1,:);
27 Ey = coords(2,:);
28 nnodes = size(coords,2);
29 nelem = size(triangle,2);
30 Edof = zeros(nelem, size(triangle,1)*2+1);
31
32 for i = 1:nelem
33
34     dofx = 2*triangle(:,i)-1;
35     dofy = 2*triangle(:,i);
36     Edof(i,1)=i;
37     Edof(i,2:2:end) = dofx;
38     Edof(i,3:2:end) = dofy;
39
40 end
41
42 %% Element Routine
43
44 ptype = 2;
45 ep = [ptype thickness];
46 ndof = 2*nnodes;
47 D = hooke(ptype,E,nu);
48 S = sparse(ndof,ndof);
49 M = sparse(ndof,ndof);
50 F = zeros(ndof,1);
51
52 for ielem = 1:size(Edof,1)
53
54     enodes = triangle(:,ielem);
55     ex = Ex(enodes);

```

```

56     ey = Ey(enodes);
57     tri_coords = [ex;ey;1,1,1]';
58     Ae = 0.5*abs(det(tri_coords));
59     [Ke,~] = plante(ex,ey,ep,D);
60     Me = Ae*rho/3*eye(6);
61     edof = Edof(ielem,2:end);
62     S(edof,edof) = S(edof,edof)+Ke;
63     M(edof,edof) = M(edof,edof) + Me;
64
65 end
66
67 %% Boundary Conditions
68
69 fixed_dofs = [];
70 left_edge = find(edges(end,:) == 5);
71
72 for le = 1:size(left_edge,2)
73     n1 = edges(1,left_edge(1,le));
74     n2 = edges(2,left_edge(1,le));
75     fixed_dofs = [fixed_dofs, 2*n1-1, 2*n2-1];
76 end
77
78 fixdof = unique(fixed_dofs);
79
80 %% Defining Contact Nodes
81
82 Contact_id = [3,4];
83 top_right_edge = find(edges(end,:) == Contact_id(1));
84 top_left_edge = find(edges(end,:) == Contact_id(2));
85 top_right_nodes = reshape(edges(1:end-1,top_right_edge),1,[]);
86 top_left_nodes = reshape(edges(1:end-1,top_left_edge),1,[]);
87 C_nodes = unique([top_right_nodes,top_left_nodes]);
88 num_candidate = numel(C_nodes);
89 active_set = false(num_candidate,1);
90
91 %% Initiations before loop
92
93 v0 = zeros(ndof,1);
94 v0(2:2:end) = v_ini;
95 v0(fixdof) = 0;
96 u0 = zeros(ndof,1);
97 n_s = 250;
98 U = zeros(ndof,n_s);
99 V = zeros(ndof,n_s);
100 int_st = zeros(n_s,1);
101 kin_eng = zeros(n_s,1);
102 T_s = linspace(0,T,n_s);
103 force_hist = zeros(n_s,1);
104
105 %% Initial Gap
106 g0 = zeros(num_candidate,1);
107 N0 = zeros(2,num_candidate);
108
109 for ic = 1:num_candidate
110
111     ni = C_nodes(ic);
112     xi = coords(:,ni);
113     u_prev = [u0(2*ni-1);u0(2*ni)];

```

```

114     [g0(ic),N0(:,ic)]= gap_lin(x_c,xi,R,u_prev);
115
116 end
117
118 %% Time Loop
119
120 for t = 2:n_s+1
121
122     u_old = u0;
123     v_old = v0;
124     F_eff = F+((2/(dt^2))*M-0.5*S)*u_old+((2/dt)*M)*v_old;
125     S_eff = ((2/(dt^2))*M+0.5*S);
126     active_local=active_set;
127     bool = true;
128     k= 0 ;
129
130     while k ==0 || bool
131
132         nC = sum(active_local);
133         index = find(active_local);
134         C_c = zeros(nC,ndof);
135         G_c = zeros(nC,1);
136         node_list = zeros(nC,1);
137         if nC~=0
138             for j = 1:nC
139                 idx = index(j);
140                 ni = C_nodes(idx);
141                 C_i = zeros(1,ndof);
142                 C_i(1,2*ni-1)= N0(1,idx);
143                 C_i(1,2*ni) = N0(2,idx);
144                 C_c(j,:) = C_i;
145                 G_c(j) = g0(idx);
146                 node_list(j)= ni;
147             end
148
149             total_dof = ndof+nC;
150             u = zeros(total_dof,1);
151             freedof = setdiff((1:total_dof)',fixdof);
152             z = sparse(nC,nC);
153             K = [S_eff , C_c';
154                 C_c , z];
155             Force = [F_eff;-G_c+C_c*u_old];
156             u(freedof)= K(freedof,freedof)\Force(freedof);
157             lambda = u(ndof+1:end);
158             u_n = u(1:ndof);
159
160         else
161             u = zeros(ndof,1);
162             freedof = setdiff((1:ndof)',fixdof);
163             u(freedof)= S_eff(freedof,freedof)\F_eff(freedof);
164             lambda = [];
165             u_n = u;
166         end
167         k= k+1;
168         new_active_local = active_local;
169         if ~isempty(lambda)
170             for l = 1:nC
171                 idx = index(l);

```

```

172         if lambda(1) <= 0
173             new_active_local(idx)=true;
174         else
175             new_active_local(idx)= false;
176         end
177     end
178 end
179 g_new = zeros(num_candidate,1);
180 N_new = zeros(2,num_candidate);
181 for j= 1:num_candidate
182     ni = C_nodes(j);
183     xi = coords(:,ni);
184     u_g = [u_n(2*ni-1);u_n(2*ni)];
185     u_prev = [u_old(2*ni-1);u_old(2*ni)];
186     [gx,Nx]= gap_lin(x_c,xi,R,u_prev);
187     g_new(j)= gx+Nx'*u_g;
188     N_new(:,j)=Nx;
189     if g_new(j) < 0
190
191         new_active_local(j)=true;
192
193     end
194 end
195 bool = any(new_active_local~=active_local);
196 active_local= new_active_local;
197 end
198
199 u0 = u_n;
200 for j = 1:num_candidate
201     ni = C_nodes(j);
202     xi = coords(:,ni);
203     u_conv = [u0(2*ni-1); u0(2*ni)];
204     [gx, N0(:,j)] = gap_lin(x_c, xi, R, u_conv);
205     g0(j)= gx+N0(:,j)'*u_conv;
206 end
207 U(:,t-1)= u0;
208 active_set= new_active_local;
209 v0= (2/(dt))*(u0-u_old)-v_old;
210 V(:,t-1)= v0;
211 int_st(t-1,1)=0.5*u0'*S*u0;
212 kin_eng(t-1,1)=0.5*v0'*M*v0;
213 if ~isempty(lambda)
214     Ny = zeros(numel(lambda),1);
215     for il = 1:numel(lambda)
216         idx = index(il);
217         Ny(il)= N0(2,idx);
218     end
219     F_contact = sum(lambda.*Ny);
220 else
221     F_contact = 0;
222 end
223 force_hist(t-1)= F_contact;
224 end
225
226 %% Plotting
227
228 % Force History
229

```

```

230 figure
231 plot(T_s,force_hist)
232 xlabel ('Indentation [in mm]')
233 ylabel ('Force [in N]')
234 title('Total Contact Force v/s Indentation')
235
236 % Simulation, Comment it for Faster run
237
238 % scale=1;
239 % frame_pause = 0.02;
240 % vert = coords';
241 % Face = triangle';
242 % figure
243 % h.patch=patch('Faces', Face, 'Vertices', vert, ...
244 %               'FaceColor','none', ...           % no face fill
245 %               'EdgeColor',[0 0 0], ...         % black mesh lines
246 %               'LineWidth',0.5);
247 %
248 % axis equal;hold on;
249 % theta = linspace(-pi/6, -pi/2, 200);
250 % xc = x_c(1); yc = x_c(2);
251 % xarc = xc + R*cos(theta);
252 % yarc = yc + R*sin(theta);
253 % h.arc = plot(xarc, yarc, 'r', 'LineWidth', 1.5);
254 % xlabel('x (mm)'); ylabel('y (mm)');
255 % title('Deformed mesh (edges only)');
256 % drawnow;
257 %
258 % for i= 1:n_s
259 %     uk = U(:,i);
260 %     ux = uk(1:2:end);
261 %     uy = uk(2:2:end);
262 %     vertk= coords'+scale.*[ux, uy];
263 %     set(h.patch,'Vertices',vertk);
264 %     title(sprintf('Time step %d / %d', i, n_s));
265 %     drawnow;
266 %     pause(frame_pause)
267 % end
268
269 % Energy over Time
270
271 figure
272 plot(T_s,int_st,LineWidth=1.5,DisplayName='Internal Strain Energy');
273 hold on
274 plot(T_s,kin_eng,LineWidth=1.5,DisplayName='Kinetic Energy');
275 plot(T_s,int_st+kin_eng,LineWidth=1.5,DisplayName='Total Energy');
276 xlabel ('Time [in sec]')
277 ylabel ('Energy [in mJ]')
278 legend (Location="best")
279 title('Development of Energy over Time')
280
281 %Deformation of Block for Selected Time Step
282
283 time_step =round([1e-3/dt,2e-3/dt,3e-3/dt,4e-3/dt,5e-3/dt]);
284 scale=1;
285 vert = coords';
286 Face = triangle';
287 figure

```

```

288 patch('Faces', Face, 'Vertices', vert, 'FaceColor', 'none', 'EdgeColor', [0
    0 0], 'LineWidth', 0.5);
289 axis equal; hold on;
290 theta = linspace(-pi/6, -pi/2, 200);
291 xc = x_c(1); yc = x_c(2);
292 xarc = xc + R*cos(theta);
293 yarc = yc + R*sin(theta);
294 plot(xarc, yarc, 'r', 'LineWidth', 1.5);
295 xlabel('x (mm)'); ylabel('y (mm)');
296 title('Initial Mesh at Time 0 ms')
297
298 for i=1:numel(time_step)
299
300     k = time_step(i);
301     uk = U(:,k);
302     ux = uk(1:2:end);
303     uy = uk(2:2:end);
304     vertk = coords' + scale.*[ux, uy];
305     figure
306     patch('Faces', Face, 'Vertices', vertk, 'FaceColor', 'none', 'EdgeColor',
    ', [0 0 0], 'LineWidth', 0.5);
307     axis equal; hold on;
308     theta = linspace(-pi/6, -pi/2, 200);
309     xc = x_c(1); yc = x_c(2);
310     xarc = xc + R*cos(theta);
311     yarc = yc + R*sin(theta);
312     plot(xarc, yarc, 'r', 'LineWidth', 1.5);
313     xlabel('x (mm)'); ylabel('y (mm)');
314     title(sprintf('Deformed mesh at Time %d ms', k*dt*1000))
315
316 end

```

B.3 Explicit Dynamic

```

1 clear all
2 close all
3 clc
4
5 %% Material Properties
6
7 E = 50e3; %Mpa
8 nu = 0.3;
9 rho = 8e-6; %kg/mm3
10 T = 5e-3; %sec
11 n_s = 50000;
12 dt= T/n_s;
13 v_ini = 10e3; %mm/sec
14 edge_length = 2; %mm
15 thickness = 1; % Assumed
16 R = 40; %mm
17 x_c = [0;60];
18 H = 20; %mm
19 L = 2*H; %mm
20
21 %% Meshing
22 ny = H/edge_length;
23 order = 1;
24 [coords, edges, triangle] = mesh(ny,order,false);
25 coords = 1000.*coords;
26 Ex = coords(1,:);
27 Ey = coords(2,:);
28 nnodes = size(coords,2);
29 nelem = size(triangle,2);
30 Edof = zeros(nelem, size(triangle,1)*2+1);
31
32 for i = 1:nelem
33
34     dofx = 2*triangle(:,i)-1;
35     dofy = 2*triangle(:,i);
36     Edof(i,1)=i;
37     Edof(i,2:2:end) = dofx;
38     Edof(i,3:2:end) = dofy;
39
40 end
41
42 %% Element Routine
43
44 ptype = 2;
45 ep = [ptype thickness];
46 ndof = 2*nnodes;
47 D = hooke(ptype,E,nu);
48 S = sparse(ndof,ndof);
49 M = sparse(ndof,ndof);
50 F = zeros(ndof,1);
51
52 for ielem = 1:size(Edof,1)
53
54     enodes = triangle(:,ielem);
55     ex = Ex(enodes);

```



```

56     ey = Ey(enodes);
57     tri_coords = [ex;ey;1,1,1]';
58     Ae = 0.5*abs(det(tri_coords));
59     [Ke,~] = plante(ex,ey,ep,D);
60     Me = Ae*rho/3*eye(6);
61     edof = Edof(ielem,2:end);
62     S(edof,edof) = S(edof,edof)+Ke;
63     M(edof,edof) = M(edof,edof) + Me;
64
65 end
66
67 MLinv = sparse(ndof, ndof);
68 for i = 1:ndof
69     MLinv(i,i) = 1 / M(i,i);
70 end
71
72 %% Boundary Conditions
73
74 fixed_dofs = [];
75 left_edge = find(edges(end,:) == 5);
76
77 for le = 1:size(left_edge,2)
78     n1 = edges(1,left_edge(1,le));
79     n2 = edges(2,left_edge(1,le));
80     fixed_dofs = [fixed_dofs, 2*n1-1, 2*n2-1];
81 end
82
83 fixdof = unique(fixed_dofs);
84 freedof = setdiff((1:ndof)', fixdof);
85
86 %% Defining Contact Nodes
87
88 Contact_id = [3,4];
89 top_right_edge = find(edges(end,:) == Contact_id(1));
90 top_left_edge = find(edges(end,:) == Contact_id(2));
91 top_right_nodes = reshape(edges(1:end-1, top_right_edge), 1, []);
92 top_left_nodes = reshape(edges(1:end-1, top_left_edge), 1, []);
93 C_nodes = unique([top_right_nodes, top_left_nodes]);
94 num_candidate = numel(C_nodes);
95 active_set = false(num_candidate, 1);
96
97 %% Initiations before loop
98 v0 = zeros(ndof, 1);
99 v0(2:2:end) = v_ini;
100 v0(fixdof) = 0;
101 u0 = zeros(ndof, 1);
102 U = zeros(ndof, n_s);
103 V = zeros(ndof, n_s);
104 int_st = zeros(n_s, 1);
105 kin_eng = zeros(n_s, 1);
106 T_s = linspace(0, T, n_s);
107 force_hist = zeros(n_s, 1);
108
109 %% Initial Gap
110 g0 = zeros(num_candidate, 1);
111 N0 = zeros(2, num_candidate);
112 for ic = 1:num_candidate
113

```

```

114     ni = C_nodes(ic);
115     xi = coords(:,ni);
116     u_prev = [u0(2*ni-1);u0(2*ni)];
117     [g0(ic),N0(:,ic)]= gap_lin(x_c,xi,R,u_prev);
118
119 end
120 utr = zeros(ndof,1);
121
122 %% Time Loop
123
124 for t = 2:n_s+1
125
126     u_old = u0;
127     v_old = v0;
128
129     utr = (u_old)+(dt.*v_old)+((dt^2)/2)*MLinv*(F-S*u_old);
130     utr(fixdof)=0;
131     active_local=active_set;
132     bool = true;
133     k= 0 ;
134     while k ==0 || bool
135
136         nC = sum(active_local);
137         index = find(active_local);
138         C_c = zeros(nC,ndof);
139         G_c = zeros(nC,1);
140         node_list = zeros(nC,1);
141         if nC~=0
142             for j = 1:nC
143                 idx = index(j);
144                 ni = C_nodes(idx);
145                 C_i = zeros(1,ndof);
146                 C_i(1,2*ni-1)= N0(1,idx);
147                 C_i(1,2*ni) = N0(2,idx);
148                 C_c(j,:) = C_i;
149                 G_c(j) = g0(idx);
150                 node_list(j)= ni;
151             end
152
153             B_eff = 0.5*dt^2*C_c*MLinv*C_c';
154             rhs = G_c+C_c*(utr-u_old);
155             lambda = B_eff\rhs;
156             u_n = utr - 0.5*(dt^2)*MLinv*C_c'*lambda;
157             u_n(fixdof)=0;
158
159         else
160
161             u_n=utr;
162             lambda =[];
163         end
164         k= k+1;
165         new_active_local = active_local;
166         if ~isempty(lambda)
167
168             for l = 1:size(index,1)
169                 idx = index(l);
170                 if lambda(l)<=0
171                     new_active_local(idx)=true;

```

```

172         else
173             new_active_local(idx)= false;
174         end
175     end
176 else
177     new_active_local=false(num_candidate,1);
178 end
179
180 g_new = zeros(num_candidate,1);
181 N_new = zeros(2,num_candidate);
182 for j= 1:num_candidate
183     ni = C_nodes(j);
184     xi = coords(:,ni);
185     u_g = [u_n(2*ni-1);u_n(2*ni)];
186     u_prev = [u_old(2*ni-1);u_old(2*ni)];
187     [gx,Nx]= gap_lin(x_c,xi,R,u_prev);
188     g_new(j)= gx+Nx'*u_g;
189     N_new(:,j)=Nx;
190     if g_new(j) < -1e-14
191         new_active_local(j)=true;
192     end
193 end
194 bool = any(new_active_local~=active_local);
195 active_local= new_active_local;
196 end
197 u0 = u_n;
198
199 for j = 1:num_candidate
200     ni = C_nodes(j);
201     xi = coords(:,ni);
202     u_conv = [u0(2*ni-1); u0(2*ni)];
203     [gx, N0(:,j)] = gap_lin(x_c, xi, R, u_conv);
204     g0(j)= gx+N0(:,j)'*u_conv;
205 end
206
207 U(:,t-1)= u0;
208 v0= (2/(dt))*(u0-u_old)-v_old;
209 v0(fixdof)=0;
210 active_set= new_active_local;
211 V(:,t-1)= v0;
212 int_st(t-1,1)=0.5*u0'*S*u0;
213 kin_eng(t-1,1)=0.5*v0'*M*v0;
214 if ~isempty(lambda)
215
216     Ny = zeros(numel(lambda),1);
217     for il = 1:numel(lambda)
218         idx = index(il);
219         Ny(il)= N0(2,idx);
220     end
221     F_contact = sum(lambda.*Ny);
222 else
223     F_contact = 0;
224 end
225 force_hist(t-1)= F_contact;
226 end
227
228 %% Plotting
229

```

```

230 % Simulation, Comment it for Faster run
231
232 % scale=1;
233 % frame_pause = 0;
234 % vert = coords';
235 % Face = triangle';
236 % figure
237 % h.patch=patch('Faces', Face, 'Vertices', vert, ...
238 %               'FaceColor','none', ...           % no face fill
239 %               'EdgeColor',[0 0 0], ...         % black mesh lines
240 %               'LineWidth',0.5);
241 % axis equal;hold on;
242 % theta = linspace(-pi/6, -pi/2, 200);
243 % xc = x_c(1); yc = x_c(2);
244 % xarc = xc + R*cos(theta);
245 % yarc = yc + R*sin(theta);
246 % h.arc = plot(xarc, yarc, 'r', 'LineWidth', 1.5);
247 % xlabel('x (mm)'); ylabel('y (mm)');
248 % title('Deformed mesh (edges only)');
249 % drawnow;
250 % for i= 1:n_s
251 %     uk = U(:,i);
252 %     ux = uk(1:2:end);
253 %     uy = uk(2:2:end);
254 %     vertk= coords'+scale.*[ux, uy];
255 %     set(h.patch,'Vertices',vertk);
256 %     title(sprintf('Time step %d / %d', i, n_s));
257 %     drawnow;
258 %     pause(frame_pause)
259 % end
260
261 % Energy Over Time
262
263 figure
264 plot(T_s,int_st,LineWidth=1.5,DisplayName='Internal Strain Energy');
265 hold on
266 plot(T_s,kin_eng,LineWidth=1.5,DisplayName='Kinetic Energy');
267 plot(T_s,int_st+kin_eng,LineWidth=1.5,DisplayName='Total Energy');
268 xlabel ('Time [in sec]')
269 ylabel ('Energy [in mJ]')
270 legend (Location="best")
271 title('Development of Energy over Time')
272
273 % Force History
274
275 figure
276 plot(T_s, force_hist)
277 xlabel ('Indentation [in mm]')
278 ylabel ('Force [in N]')
279 title('Total Contact Force v/s Indentation')
280
281 % Smoothened Force History
282
283 figure
284 plot(T_s, smoothdata(force_hist, 'movmean', 100))
285 xlabel ('Indentation [in mm]')
286 ylabel ('Force [in N]')
287 title('Smoothened Total Contact Force v/s Indentation')

```

```

288
289 %Deformation of Block for Selected Time Step
290
291 time_step =round([1e-3/dt,2e-3/dt,3e-3/dt,4e-3/dt,5e-3/dt]);
292 scale=1;
293 vert = coords';
294 Face = triangle';
295 figure
296 patch('Faces', Face, 'Vertices', vert, 'FaceColor', 'none', 'EdgeColor', [0
    0 0], 'LineWidth', 0.5);
297 axis equal;hold on;
298 theta = linspace(-pi/6, -pi/2, 200);
299 xc = x_c(1); yc = x_c(2);% bottom 180 degrees
300 xarc = xc + R*cos(theta);
301 yarc = yc + R*sin(theta);
302 plot(xarc, yarc, 'r', 'LineWidth', 1.5);
303 xlabel('x (mm)'); ylabel('y (mm)');
304 title ('Initial Mesh at Time 0 ms')
305
306 for i=1:numel(time_step)
307
308     k = time_step(i);
309     uk = U(:,k);
310     ux = uk(1:2:end);
311     uy = uk(2:2:end);
312     vertk= coords'+scale.*[ux, uy];
313     figure
314     patch('Faces', Face, 'Vertices', vertk, 'FaceColor', 'none', 'EdgeColor
    ', [0 0 0], 'LineWidth', 0.5);
315     axis equal;hold on;
316     theta = linspace(-pi/6, -pi/2, 200);
317     xc = x_c(1); yc = x_c(2);% bottom 180 degrees
318     xarc = xc + R*cos(theta);
319     yarc = yc + R*sin(theta);
320     plot(xarc, yarc, 'r', 'LineWidth', 1.5);
321     xlabel('x (mm)'); ylabel('y (mm)');
322     title (sprintf('Deformed mesh at Time %d ms',k*dt*1000))
323
324 end

```

C

Appendix: Supporting Function

C.1 Gap Function

```
1 function [g,N]= gap_lin(X_bar,X,R,u)
2 a = norm((X+u)-X_bar);
3 b = (X+u)-X_bar;
4 N = b/a;
5 g = a-R-N'*u;
6
7 end
```